

Hướng dẫn thực hiện bài tập lớn cho sinh viên KHMT 17-01

Bài tập lớn học phần CSC4005 – Học sâu

ThS. Lê Thị Thùy Trang

2026-04-25

1 Giới thiệu bài tập lớn

1.1 Bối cảnh

Bài tập lớn học phần **CSC4005 – Học sâu** được thiết kế theo định hướng **mini-capstone**: sinh viên không chỉ chạy mô hình để có một con số kết quả, mà phải xây dựng một pipeline học sâu có thể tái lập, gồm dữ liệu, baseline, mô hình học sâu, thí nghiệm W&B, đánh giá định lượng, phân tích lỗi và demo.

Điểm nhấn của bài tập lớn là tính **đa dạng dữ liệu**. Các đề tài không chỉ giới hạn ở ảnh thông thường mà mở rộng sang nhiều kiểu dữ liệu khác nhau như ảnh nông nghiệp, ảnh y tế, ảnh vệ tinh, dữ liệu bảng trong an toàn thông tin, dữ liệu cảm biến, dữ liệu đa phương thức, ảnh công nghiệp, ảnh phân loại rác thải, tín hiệu y sinh ECG và đặc trưng tĩnh của file phục vụ phát hiện mã độc.

Thông qua bài tập lớn, sinh viên cần chứng minh được rằng mình hiểu quy trình xây dựng một hệ thống học sâu từ đầu đến cuối: từ xác định bài toán, xử lý dữ liệu, thiết kế mô hình, huấn luyện, theo dõi thí nghiệm, đánh giá, phân tích lỗi, export mô hình sang ONNX, kiểm thử suy luận bằng `onnxruntime` đến xây dựng demo. Với nhóm khá/giỏi, sinh viên có thể thực hiện thêm model compression để tối ưu kích thước mô hình hoặc tốc độ suy luận.

1.2 Nguyên tắc thiết kế đề tài

Một đề tài hợp lệ cần thỏa mãn các nguyên tắc sau:

- Có **dataset rõ ràng** hoặc có quy trình tự xây dựng dữ liệu hợp lệ.
- Nguồn dữ liệu phải **gắn trực tiếp với bài toán**; không dùng dataset chỉ để liệt kê cho có.
- Có **baseline** để so sánh với mô hình học sâu.
- Có **mô hình học sâu** được huấn luyện, fine-tune hoặc xây dựng phù hợp với kiểu dữ liệu.
- Có **W&B tracking** để log thí nghiệm, metric, hyperparameter và artifact.
- Có **ONNX deployment** ở mức tối thiểu: export mô hình tốt nhất sang `.onnx` và chạy suy luận bằng `onnxruntime`.
- Có **phân tích lỗi** dựa trên mẫu dự đoán thật.
- Khuyến khích thực hiện **model compression** như quantization, pruning, knowledge distillation hoặc sử dụng mô hình nhẹ để cải thiện khả năng triển khai.
- Có **demo chạy được** và báo cáo kỹ thuật cuối kỳ.

Sinh viên phải mô tả rõ nguồn dữ liệu, cấu trúc dữ liệu, cách chia tập, các bước tiền xử lý và giới hạn của dữ liệu trong báo cáo.

Nhóm không được chỉ xây dựng demo gọi API hoặc dùng mô hình có sẵn mà không huấn luyện/đánh giá. Bắt buộc phải có dữ liệu, baseline, mô hình học sâu, thí nghiệm được log bằng W&B, đánh giá định lượng, phân tích lỗi định tính và file ONNX chạy được bằng `onnxruntime`.

2 Mục tiêu học tập

Sau khi hoàn thành bài tập lớn, sinh viên có thể:

1. Xác định được một bài toán học sâu rõ ràng, có đầu vào, đầu ra và metric phù hợp.
2. Tìm kiếm, lựa chọn hoặc xây dựng dataset phù hợp với bài toán.
3. Xây dựng pipeline tiền xử lý dữ liệu cho từng kiểu dữ liệu khác nhau.
4. Xây dựng baseline và giải thích vai trò của baseline.
5. Thiết kế, huấn luyện và tối ưu mô hình học sâu.
6. Sử dụng W&B để log hyperparameters, loss, metrics, artifacts và so sánh runs.
7. Đánh giá mô hình bằng metric phù hợp.
8. Phân tích lỗi dựa trên mẫu dự đoán thật.
9. Export mô hình học sâu tốt nhất sang định dạng ONNX và kiểm thử suy luận bằng `onnxruntime`.
10. Đo và so sánh thời gian suy luận của mô hình gốc và mô hình ONNX.
11. Giải thích được ý nghĩa của model compression; nếu thực hiện, biết so sánh trước–sau về chất lượng mô hình, kích thước và tốc độ suy luận.
12. Xây dựng demo đơn giản cho hệ thống.
13. Viết báo cáo kỹ thuật có bằng chứng và có khả năng tái lập.

3 Quy định chung

3.1 Tổ chức nhóm

- Mỗi nhóm gồm 2–4 sinh viên.
- Mỗi nhóm chọn một đề tài trong danh sách đề tài bên dưới.
- Một đề tài nên có tối đa 1 nhóm chọn.

3.2 Mức tối thiểu để được nghiệm thu

Một nhóm chỉ được coi là hoàn thành nếu có đủ:

1. Dataset hoặc nguồn dữ liệu rõ ràng, đúng với đề tài.
2. Baseline model.
3. Deep learning model hoặc mô hình học sâu tự xây/fine-tune.
4. Tối thiểu 5 W&B runs.
5. Bảng kết quả định lượng.
6. Phân tích lỗi tối thiểu 10 mẫu.

7. File mô hình tốt nhất ở định dạng gốc, ví dụ .pt hoặc .pth.
8. File mô hình ONNX, ví dụ best_model.onnx.
9. Script kiểm thử suy luận ONNX bằng onnxruntime.
10. Demo chạy được.
11. Báo cáo cuối kỳ.
12. Link GitHub repository.

3.3 Cấu trúc thư mục khuyến nghị

```
project-name/  
  README.md  
  requirements.txt  
  configs/  
    baseline.yaml  
    model.yaml  
  data/  
    raw/  
    processed/  
    splits/  
      train.csv  
      dev.csv  
      test.csv  
  notebooks/  
  src/  
    preprocess.py  
    train_baseline.py  
    train_model.py  
    evaluate.py  
    export_onnx.py  
    infer.py  
    infer_onnx.py  
    benchmark_inference.py  
    app.py  
  outputs/  
    metrics/  
    figures/  
    predictions/  
    models/  
      best_model.pt  
      best_model.onnx  
  reports/  
    experiment_log.md  
    error_analysis.md  
    onnx_deployment.md  
    compression_report.md  # nếu có thực hiện model compression  
    final_report.pdf  
  wandb_report_link.txt
```

3.4 Cài đặt môi trường tham khảo

```
conda create -n csc4005-project python=3.10 -y
conda activate csc4005-project
pip install -r requirements.txt
wandb login
```

Một file `requirements.txt` tối thiểu có thể gồm:

```
torch
torchvision
torchaudio
transformers
datasets
scikit-learn
pandas
numpy
matplotlib
opencv-python
pillow
wandb
onnx
onnxruntime
streamlit
gradio
```

Tùy đề tài, nhóm có thể bổ sung thêm:

```
torch-geometric
wfdb
librosa
timm
albumentations
pytorch-lightning
```

4 Danh sách đề tài

Bảng sau chỉ liệt kê dataset chính gắn với từng đề tài. Chi tiết yêu cầu nằm ở các mục tiếp theo.

STT	Đề tài	Dataset / nguồn dữ liệu chính
1	Phân loại bệnh trên lá cây bằng ảnh và giải thích mô hình bằng Grad-CAM	PlantVillage Dataset.
2	Phát hiện tấn công mạng từ dữ liệu network flow	UNSW-NB15 hoặc CICIDS2017.
3	Nhận diện hoạt động con người từ cảm biến điện thoại	UCI Human Activity Recognition Using Smartphones.
4	Phân loại ảnh vệ tinh phục vụ giám sát sử dụng đất	EuroSAT.
5	Phát hiện tin giả đa phương thức từ văn bản và ảnh	Fakeddit.
6	Phân loại tổn thương da bằng ảnh y tế	HAM10000.
7	Phát hiện lỗi sản phẩm công nghiệp bằng học sâu	MVTec AD.

8	Phân loại rác thải bằng ảnh phục vụ môi trường xanh	TrashNet hoặc Garbage Classification Dataset.
9	Phân loại rối loạn nhịp tim từ tín hiệu ECG	MIT-BIH Arrhythmia Database.
10	Phân loại mã độc hoặc file độc hại dựa trên đặc trưng tĩnh	EMBER Dataset.

4.1 Đề tài 1. Phân loại bệnh trên lá cây bằng ảnh và giải thích mô hình bằng Grad-CAM

4.1.1 Mục tiêu

Xây dựng mô hình học sâu có khả năng phân loại ảnh lá cây thành các nhóm bệnh khác nhau hoặc lá khỏe mạnh. Đề tài phù hợp với hướng **AI trong nông nghiệp thông minh**. Ngoài việc dự đoán nhãn, nhóm cần sử dụng Grad-CAM hoặc phương pháp tương tự để trực quan hóa vùng ảnh mà mô hình tập trung vào.

4.1.2 Dữ liệu

Dataset khuyến nghị: **PlantVillage Dataset**.

Nguồn dữ liệu:

- Kaggle PlantVillage: [Link](#).

Dữ liệu gồm ảnh lá cây của nhiều loại cây trồng như cà chua, khoai tây, táo, nho, ngô. Nhóm có thể chọn toàn bộ dataset hoặc chọn một nhóm cây cụ thể để giảm độ khó.

4.1.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

```
Ảnh lá cây
→ resize/normalize/augmentation
→ CNN baseline
→ transfer learning
→ đánh giá mô hình
→ Grad-CAM visualization
```

Nhóm phải có:

- Baseline CNN đơn giản;
- Mô hình chính: ResNet, EfficientNet hoặc MobileNet;
- Data augmentation: flip, rotation, color jitter hoặc random crop;
- Confusion matrix;
- Grad-CAM cho ít nhất 10 ảnh dự đoán đúng và 10 ảnh dự đoán sai.

4.1.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. So sánh mô hình huấn luyện từ đầu và transfer learning.
2. Xử lý mất cân bằng lớp bằng weighted loss hoặc sampling.

3. Kiểm tra robustness với ảnh bị nhiễu, mờ hoặc thay đổi ánh sáng.
4. Xây dựng demo upload ảnh và trả kết quả dự đoán.
5. So sánh Grad-CAM giữa mô hình tốt và mô hình yếu.

4.1.5 W&B và đánh giá

Tối thiểu 5 runs:

- CNN baseline;
- ResNet chưa tuning;
- ResNet có augmentation;
- EfficientNet hoặc MobileNet;
- Best model run.

Metric bắt buộc:

- Accuracy;
- Macro-F1;
- Precision/Recall theo từng lớp;
- Confusion matrix;
- Bảng 10 mẫu dự đoán sai.

4.1.6 Sản phẩm nộp

```
- train_baseline.py
- train_model.py
- evaluate.py
- gradcam_visualization.py
- error_analysis.md
- metrics_summary.csv
- wandb_report_link.txt
- demo_app.py hoặc demo_notebook.ipynb
- final_report.pdf
```

4.2 Đề tài 2. Phát hiện tấn công mạng từ dữ liệu network flow

4.2.1 Mục tiêu

Xây dựng mô hình học sâu để phân loại lưu lượng mạng bình thường và lưu lượng tấn công. Đề tài phù hợp với hướng **deep learning for cybersecurity**. Nhóm có thể triển khai bài toán nhị phân hoặc bài toán đa lớp theo loại tấn công.

4.2.2 Dữ liệu

Nguồn dữ liệu gợi ý:

- UNSW-NB15: [Link](#).

- **CICIDS2017**: [Link](#).

Nhóm nên chọn một dataset chính. Nếu dataset quá lớn, nhóm có thể lấy mẫu hợp lý nhưng phải ghi rõ cách lấy mẫu.

4.2.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

```
Network flow features
→ xử lý missing values
→ encoding categorical features
→ normalization
→ baseline
→ deep learning model
→ đánh giá binary/multiclass
```

Nhóm phải có:

- Baseline: Logistic Regression, Random Forest hoặc XGBoost;
- Mô hình học sâu: MLP, AutoEncoder hoặc TabTransformer;
- Xử lý mất cân bằng lớp;
- Đánh giá riêng cho lớp attack;
- Phân tích các loại tấn công để nhầm.

4.2.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. So sánh binary classification và multi-class classification.
2. Dùng AutoEncoder để phát hiện bất thường.
3. Thử TabTransformer hoặc mô hình chuyên cho dữ liệu bảng.
4. Phân tích feature importance hoặc embedding learned features.
5. Xây dựng demo nhập một flow và trả cảnh báo.

4.2.5 W&B và đánh giá

Metric bắt buộc:

- Accuracy;
- Precision;
- Recall;
- Macro-F1;
- ROC-AUC nếu là bài toán nhị phân;
- Confusion matrix.

4.2.6 Sản phẩm nộp

```
- preprocess_network_flow.py
- train_baseline.py
- train_model.py
- evaluate.py
- test_predictions.csv
- error_analysis.md
- metrics_summary.csv
- wandb_report_link.txt
- demo_app.py
- final_report.pdf
```

4.3 Đề tài 3. Nhận diện hoạt động con người từ cảm biến điện thoại

4.3.1 Mục tiêu

Xây dựng mô hình phân loại hoạt động của con người như đi bộ, đứng, ngồi, nằm, đi lên cầu thang, đi xuống cầu thang dựa trên dữ liệu cảm biến điện thoại. Đây là đề tài giúp sinh viên làm quen với **dữ liệu cảm biến và chuỗi thời gian đa biến**.

4.3.2 Dữ liệu

Dataset khuyến nghị: **UCI Human Activity Recognition Using Smartphones**.

Nguồn dữ liệu:

- UCI HAR Dataset: [Link](#).

Dữ liệu gồm tín hiệu gia tốc kế và con quay hồi chuyển thu từ điện thoại thông minh.

4.3.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

```
Sensor signals
→ chuẩn hóa tín hiệu
→ chia window
→ baseline MLP
→ 1D-CNN/LSTM/GRU
→ đánh giá theo từng hoạt động
```

Nhóm phải có:

- Baseline MLP hoặc Random Forest;
- Mô hình chính: 1D-CNN, LSTM hoặc GRU;
- Phân tích confusion matrix;
- So sánh ít nhất 2 kiến trúc;
- Phân tích các cặp hoạt động dễ nhầm.

4.3.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. Thử Transformer cho dữ liệu cảm biến.
2. So sánh dùng feature có sẵn và dùng raw signal.
3. Thử thay đổi window size.
4. Trực quan hóa embedding bằng PCA/t-SNE.
5. Xây dựng demo nhận file tín hiệu và phân loại hoạt động.

4.3.5 W&B và đánh giá

Metric bắt buộc:

- Accuracy;
- Macro-F1;
- Per-class F1;
- Confusion matrix;
- Bảng 10 mẫu dự đoán sai.

4.3.6 Sản phẩm nộp

```
- preprocess_sensor.py
- train_mlp.py
- train_sequence_model.py
- evaluate.py
- error_analysis.md
- metrics_summary.csv
- wandb_report_link.txt
- demo_notebook.ipynb
- final_report.pdf
```

4.4 Đề tài 4. Phân loại ảnh vệ tinh phục vụ giám sát sử dụng đất

4.4.1 Mục tiêu

Xây dựng mô hình phân loại ảnh vệ tinh thành các loại hình sử dụng đất như khu dân cư, rừng, sông, đường cao tốc, khu công nghiệp, đồng cỏ, cây trồng. Đề tài phù hợp với hướng **AI cho viễn thám và môi trường**.

4.4.2 Dữ liệu

Dataset khuyến nghị: **EuroSAT**.

Nguồn dữ liệu:

- EuroSAT GitHub: [Link](#).

Dataset gồm ảnh vệ tinh Sentinel-2 với nhiều lớp sử dụng đất khác nhau.

4.4.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

```
Ảnh vệ tinh
→ resize/normalize/augmentation
→ CNN baseline
→ transfer learning hoặc ViT
→ đánh giá theo từng lớp
```

Nhóm phải có:

- Baseline CNN đơn giản;
- Mô hình chính: ResNet, EfficientNet hoặc Vision Transformer;
- Data augmentation phù hợp;
- Phân tích các lớp dễ nhầm như residential/industrial hoặc river/highway;
- Demo upload ảnh và trả lớp dự đoán.

4.4.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. So sánh RGB và multispectral nếu nhóm khai thác được dữ liệu.
2. Dùng pretrained model trên ImageNet và so sánh với huấn luyện từ đầu.
3. Trực quan hóa Grad-CAM hoặc attention map.
4. Kiểm tra robustness với ảnh bị nhiễu hoặc thay đổi độ sáng.
5. Tạo bản đồ mini gồm nhiều ảnh tile và thống kê lớp sử dụng đất.

4.4.5 W&B và đánh giá

Metric bắt buộc:

- Accuracy;
- Macro-F1;
- Per-class accuracy;
- Confusion matrix;
- Bảng so sánh CNN, ResNet/EfficientNet và ViT nếu có.

4.4.6 Sản phẩm nộp

```
- preprocess_satellite.py
- train_cnn.py
- train_transfer_learning.py
- evaluate.py
- visualization.py
- error_analysis.md
- wandb_report_link.txt
```

- demo_app.py
- final_report.pdf

4.5 Đề tài 5. Phát hiện tin giả đa phương thức từ văn bản và ảnh

4.5.1 Mục tiêu

Xây dựng mô hình phát hiện tin giả bằng cách kết hợp thông tin từ **văn bản** và **hình ảnh**. Đây là đề tài có độ khó cao vì sinh viên phải xử lý dữ liệu đa phương thức và đánh giá xem việc kết hợp hai nguồn thông tin có thực sự cải thiện kết quả không.

4.5.2 Dữ liệu

Dataset khuyến nghị: **Fakeddit**.

Nguồn dữ liệu:

- Fakeddit GitHub: [Link](#).

Nhóm có thể chọn một phần dataset phù hợp với năng lực tính toán. Cần ghi rõ số lượng mẫu được sử dụng và cách chia tập.

4.5.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

```
Text + Image
→ text preprocessing
→ image preprocessing
→ text-only model
→ image-only model
→ multimodal fusion model
→ so sánh kết quả
```

Nhóm phải có:

- Text-only baseline;
- Image-only baseline;
- Multimodal model kết hợp embedding văn bản và embedding ảnh;
- So sánh kết quả giữa text-only, image-only và multimodal;
- Phân tích các mẫu mà một modality đúng nhưng modality còn lại sai.

4.5.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. Thử nhiều chiến lược fusion: early fusion, late fusion hoặc attention fusion.
2. Thử BERT/DistilBERT cho text và ResNet/ViT cho image.
3. Phân tích trường hợp text và image mâu thuẫn nhau.
4. Kiểm tra robustness khi thiếu ảnh hoặc thiếu văn bản.

5. Xây dựng demo nhập text và ảnh để dự đoán thật/giả.

4.5.5 W&B và đánh giá

Tối thiểu 5 runs:

- Text-only baseline;
- Image-only baseline;
- Multimodal fusion lần 1;
- Multimodal fusion lần 2 với thay đổi hyperparameter;
- Best multimodal model.

Metric bắt buộc:

- Accuracy;
- Macro-F1;
- ROC-AUC nếu là binary classification;
- Confusion matrix;
- Bảng mẫu dự đoán sai.

4.5.6 Sản phẩm nộp

```
- preprocess_text.py
- preprocess_image.py
- train_text_model.py
- train_image_model.py
- train_multimodal_model.py
- evaluate.py
- error_analysis.md
- metrics_summary.csv
- wandb_report_link.txt
- demo_app.py
- final_report.pdf
```

4.6 Đề tài 6. Phân loại tổn thương da bằng ảnh y tế

4.6.1 Mục tiêu

Xây dựng mô hình phân loại ảnh tổn thương da thành các nhóm bệnh khác nhau. Đề tài phù hợp với hướng **AI trong y tế**. Nhóm cần chú ý vấn đề mất cân bằng lớp và phải trình bày rõ ràng hệ thống chỉ phục vụ mục đích học tập, không thay thế chẩn đoán y khoa.

4.6.2 Dữ liệu

Dataset khuyến nghị: **HAM10000**.

Nguồn dữ liệu:

- HAM10000 trên Kaggle: [Link](#).

Dataset gồm ảnh tổn thương da với nhiều nhóm chẩn đoán khác nhau.

4.6.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

```
Ảnh tổn thương da
→ resize/normalize/augmentation
→ xử lý mất cân bằng lớp
→ CNN baseline
→ transfer learning
→ đánh giá y tế cơ bản
→ Grad-CAM
```

Nhóm phải có:

- Baseline CNN;
- Mô hình chính: ResNet, DenseNet hoặc EfficientNet;
- Xử lý class imbalance bằng class weights, oversampling hoặc focal loss;
- Đánh giá recall/sensitivity cho từng lớp;
- Grad-CAM cho một số mẫu dự đoán đúng và sai.

4.6.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. So sánh class weights và focal loss.
2. Calibration confidence cho mô hình.
3. Phân tích lỗi theo từng lớp bệnh.
4. Giao diện demo có cảnh báo *not for medical diagnosis*.
5. So sánh EfficientNet và DenseNet.

4.6.5 W&B và đánh giá

Metric bắt buộc:

- Accuracy;
- Macro-F1;
- Recall/Sensitivity;
- Confusion matrix;
- Per-class F1;
- Grad-CAM visualization.

4.6.6 Sản phẩm nộp

```
- preprocess_skin_images.py
- train_baseline.py
- train_transfer_model.py
- evaluate.py
- gradcam_visualization.py
- error_analysis.md
- metrics_summary.csv
- wandb_report_link.txt
- demo_app.py
- final_report.pdf
```

4.7 Đề tài 7. Phát hiện lỗi sản phẩm công nghiệp bằng học sâu

4.7.1 Mục tiêu

Xây dựng mô hình phát hiện sản phẩm lỗi trong dây chuyền sản xuất dựa trên ảnh bề mặt sản phẩm. Đây là đề tài thú vị vì không chỉ là phân loại ảnh thông thường mà còn liên quan đến **anomaly detection**.

4.7.2 Dữ liệu

Dataset khuyến nghị: **MVTec AD**.

Nguồn dữ liệu:

- MVTec AD: [Link](#).

Dữ liệu gồm nhiều nhóm sản phẩm như chai, viên thuốc, gỗ, da, kim loại, cáp, thảm, gạch, hạt, transistor.

4.7.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

```
Ảnh sản phẩm
→ resize/normalize
→ CNN normal/defect baseline
→ AutoEncoder reconstruction
→ anomaly score
→ đánh giá phát hiện lỗi
```

Nhóm phải có:

- Baseline CNN phân loại normal/defect;
- AutoEncoder hoặc mô hình reconstruction-based anomaly detection;
- So sánh supervised classification và anomaly detection;
- Trực quan hóa ảnh gốc, ảnh tái tạo và vùng sai khác;
- Phân tích các loại lỗi khó phát hiện.

4.7.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. Thử Variational AutoEncoder hoặc U-Net AutoEncoder.
2. Tạo heatmap vùng lỗi.
3. So sánh anomaly threshold khác nhau.
4. Đánh giá theo từng nhóm sản phẩm.
5. Demo upload ảnh sản phẩm và trả cảnh báo normal/defect.

4.7.5 W&B và đánh giá

Metric bắt buộc:

- Accuracy hoặc Balanced Accuracy;
- Precision;
- Recall;
- F1-score;
- ROC-AUC;
- Reconstruction error nếu dùng AutoEncoder.

4.7.6 Sản phẩm nộp

```
- preprocess_mvtec.py
- train_cnn_classifier.py
- train_autoencoder.py
- evaluate_anomaly.py
- reconstruction_visualization.py
- error_analysis.md
- metrics_summary.csv
- wandb_report_link.txt
- demo_app.py
- final_report.pdf
```

4.8 Đề tài 8. Phân loại rác thải bằng ảnh phục vụ môi trường xanh

4.8.1 Mục tiêu

Xây dựng mô hình học sâu có khả năng phân loại ảnh rác thải thành các nhóm như giấy, nhựa, kim loại, thủy tinh, bìa carton, rác hữu cơ hoặc rác hỗn hợp. Đề tài phù hợp với hướng **AI for environment** và có thể phát triển thành demo hỗ trợ phân loại rác tại nguồn.

Đề tài này không chỉ yêu cầu mô hình đạt kết quả tốt, mà còn cần nhóm phân tích được các lớp dễ nhầm, ví dụ nhựa và thủy tinh, giấy và bìa carton, hoặc ảnh có nền phức tạp.

4.8.2 Dữ liệu

Nguồn dữ liệu gợi ý:

- **TrashNet:** [Link](#).
- **Garbage Classification Dataset – Kaggle:** [Link](#).

Nhóm có thể chọn một trong hai dataset trên. Nếu muốn mở rộng, nhóm có thể tự chụp thêm ảnh rác thải trong bối cảnh thực tế, nhưng phải mô tả rõ cách thu thập, gán nhãn và chia tập dữ liệu.

4.8.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

```
Ảnh rác thải
→ resize/normalize/augmentation
→ CNN baseline
→ transfer learning
→ phân loại nhóm rác
→ phân tích lỗi theo từng lớp
```

Nhóm phải có:

- Baseline dùng CNN tự xây đơn giản;
- Mô hình chính dùng transfer learning, ví dụ ResNet, MobileNetV2, EfficientNet hoặc ConvNeXt nhỏ;
- Data augmentation phù hợp, ví dụ random crop, horizontal flip, color jitter, rotation;
- Đánh giá trên tập test tách biệt;
- Phân tích các lớp dễ nhầm và lý do nhầm lẫn;
- Demo upload ảnh và trả về loại rác dự đoán.

4.8.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. So sánh ít nhất 2 backbone transfer learning, ví dụ ResNet và MobileNetV2.
2. Dùng Grad-CAM để trực quan hóa vùng ảnh mô hình chú ý khi phân loại.
3. Thử xử lý mất cân bằng lớp bằng class weights, weighted sampler hoặc augmentation có định hướng.
4. Xây dựng chức năng top-3 prediction kèm xác suất.
5. Làm demo Streamlit/Gradio mô phỏng hệ thống hỗ trợ phân loại rác tại nguồn.

4.8.5 W&B và đánh giá

Metric bắt buộc:

- Accuracy;
- Macro F1-score;
- Precision;
- Recall;

- Confusion matrix;
- Một số ảnh dự đoán sai kèm phân tích.

Nhóm cần log lên W&B:

- Training loss và validation loss;
- Accuracy/F1-score theo từng epoch;
- Hyperparameters: learning rate, batch size, optimizer, augmentation, backbone;
- Confusion matrix;
- Bảng so sánh tối thiểu 5 runs;
- Một số ảnh dự đoán đúng/sai, ưu tiên các lớp dễ nhầm.

4.8.6 Sản phẩm nộp

```
- preprocess_garbage_images.py
- train_cnn_baseline.py
- train_transfer_learning.py
- evaluate_garbage_classifier.py
- gradcam_visualization.py nếu có
- error_analysis.md
- metrics_summary.csv
- wandb_report_link.txt
- demo_app.py hoặc demo_notebook.ipynb
- final_report.pdf
```

4.9 Đề tài 9. Phân loại rối loạn nhịp tim từ tín hiệu ECG

4.9.1 Mục tiêu

Xây dựng mô hình học sâu phân loại nhịp tim bình thường và bất thường từ tín hiệu ECG. Đây là đề tài phù hợp để sinh viên học cách xử lý **dữ liệu tín hiệu y sinh dạng chuỗi thời gian**.

4.9.2 Dữ liệu

Dataset khuyến nghị: **MIT-BIH Arrhythmia Database**.

Nguồn dữ liệu:

- MIT-BIH Arrhythmia Database: [Link](#).

Dữ liệu gồm tín hiệu ECG và annotation nhịp tim. Nhóm cần đọc kỹ cách tổ chức dữ liệu và cách trích đoạn tín hiệu.

4.9.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

ECG signal

- đọc tín hiệu bằng wfdb
- lọc/chuẩn hóa tín hiệu
- chia đoạn beat/window
- 1D-CNN baseline
- LSTM/GRU/Transformer
- đánh giá nhịp bất thường

Nhóm phải có:

- Baseline MLP hoặc 1D-CNN đơn giản;
- Mô hình chính: 1D-CNN, LSTM, GRU hoặc Transformer;
- Biểu đồ tín hiệu mẫu;
- Phân tích confusion matrix;
- Chú ý xử lý mất cân bằng giữa nhịp bình thường và nhịp bất thường.

4.9.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. So sánh phân loại beat-level và window-level.
2. Thử lọc nhiễu tín hiệu trước khi huấn luyện.
3. Dùng focal loss hoặc weighted loss.
4. Visualize attention hoặc vùng tín hiệu quan trọng.
5. Demo upload một đoạn ECG và trả kết quả dự đoán.

4.9.5 W&B và đánh giá

Metric bắt buộc:

- Accuracy;
- Macro-F1;
- Precision/Recall cho lớp bất thường;
- Confusion matrix;
- Bảng 10 mẫu dự đoán sai.

4.9.6 Sản phẩm nộp

```
- preprocess_ecg.py
- train_1dcnn.py
- train_sequence_model.py
- evaluate_ecg.py
- signal_visualization.py
- error_analysis.md
- metrics_summary.csv
- wandb_report_link.txt
```

- demo_notebook.ipynb
- final_report.pdf

4.10 Đề tài 10. Phân loại mã độc hoặc file độc hại dựa trên đặc trưng tĩnh

4.10.1 Mục tiêu

Xây dựng mô hình học sâu phân loại file là lành tính hay độc hại dựa trên các đặc trưng tĩnh. Đề tài phù hợp với hướng **deep learning for malware detection**. Sinh viên cần tập trung vào pipeline xử lý dữ liệu bảng, đánh giá lớp malware và phân tích các trường hợp false negative.

4.10.2 Dữ liệu

Dataset khuyến nghị: **EMBER**.

Nguồn dữ liệu:

- EMBER Dataset: [Link](#).

Dataset gồm đặc trưng trích xuất từ file PE. Nếu dataset quá lớn, nhóm có thể sử dụng subset nhưng phải ghi rõ cách chọn subset.

4.10.3 Yêu cầu kỹ thuật tối thiểu

Pipeline bắt buộc:

```
Static malware features
→ load/clean features
→ normalization
→ baseline ML
→ deep learning model
→ đánh giá malware detection
```

Nhóm phải có:

- Baseline: Logistic Regression, Random Forest hoặc LightGBM;
- Mô hình học sâu: MLP, AutoEncoder hoặc TabTransformer;
- Đánh giá ROC-AUC và F1-score;
- Phân tích false positive và false negative;
- Trình bày rõ giới hạn của mô hình dựa trên đặc trưng tĩnh.

4.10.4 Yêu cầu nâng cao

Chọn ít nhất 2 yêu cầu:

1. So sánh static-feature model và anomaly detection model.
2. Phân tích feature importance hoặc embedding.
3. Thử threshold khác nhau để giảm false negative.
4. Calibration confidence.

5. Demo nhập vector đặc trưng và trả kết quả benign/malware.

4.10.5 W&B và đánh giá

Metric bắt buộc:

- Accuracy;
- Precision;
- Recall;
- F1-score;
- ROC-AUC;
- Confusion matrix.

4.10.6 Sản phẩm nộp

```
- preprocess_ember.py
- train_baseline.py
- train_mlp.py
- train_tabtransformer.py
- evaluate_malware.py
- error_analysis.md
- metrics_summary.csv
- wandb_report_link.txt
- demo_app.py
- final_report.pdf
```

5 Yêu cầu về W&B cho tất cả đề tài

Mỗi nhóm phải tạo một project W&B có tên theo cấu trúc:

```
csc4005-khmt16-01-<topic-short-name>
```

Ví dụ:

```
csc4005-khmt16-01-skin-lesion-classification
```

Mỗi nhóm có tối thiểu 5 runs:

1. Baseline run.
2. Deep learning model lần 1.
3. Deep learning model lần 2 với thay đổi hyperparameter.
4. Deep learning model lần 3 với thay đổi model hoặc preprocessing.
5. Best model run.

Trong báo cáo, nhóm phải chèn bảng tổng hợp như sau:

Run	Model	Key setting	Validation metric	Test metric
Run 1	Baseline	...	...	...
Run 2	DL model	...	...	...
Run 3	DL model	...	...	...
Run 4	DL model	...	...	...
Run 5	Best model	...	...	...

Ngoài bảng trên, nhóm cần có một file `reports/experiment_log.md` ghi ngắn gọn từng lần chạy:

```
# Experiment Log
```

```
## Run 1: Baseline
```

```
- Model:
- Dataset split:
- Key hyperparameters:
- Validation result:
- Test result:
- Nhận xét:
```

```
## Run 2: ...
```

6 Yêu cầu triển khai ONNX và model compression

6.1 ONNX bắt buộc cho tất cả đề tài

Sau khi huấn luyện và lựa chọn mô hình tốt nhất, mỗi nhóm phải export mô hình sang định dạng ONNX để kiểm thử khả năng triển khai độc lập với framework huấn luyện.

Nhóm cần thực hiện tối thiểu:

1. Lưu mô hình tốt nhất ở định dạng gốc, ví dụ `best_model.pt` hoặc `best_model.pth`.
2. Export mô hình tốt nhất sang định dạng `best_model.onnx`.
3. Viết script `src/infer_onnx.py` để chạy suy luận bằng `onnxruntime`.
4. Kiểm thử ONNX trên tối thiểu 5 mẫu dữ liệu thật từ tập test.
5. So sánh kết quả dự đoán giữa mô hình gốc và mô hình ONNX.
6. Đo thời gian suy luận trung bình trên tối thiểu 20 mẫu nếu dữ liệu đủ lớn.
7. Viết file `reports/onnx_deployment.md` mô tả quy trình export, kiểm thử và nhận xét khả năng triển khai.

Với các đề tài có pipeline phức tạp, ví dụ đề tài đa phương thức text + image, nhóm có thể export phần mô hình chính hoặc một nhánh mô hình nếu việc export toàn bộ pipeline gặp khó khăn kỹ thuật. Tuy nhiên, nhóm phải giải thích rõ giới hạn này trong báo cáo.

Bảng báo cáo ONNX tối thiểu:

Phiên bản	Accuracy/F1	Kích thước	Inference time	Nhận xét
Mô hình gốc				
Mô hình ONNX				

6.2 Model compression nâng cao

Model compression là yêu cầu nâng cao, không bắt buộc với tất cả nhóm. Nhóm thực hiện tốt phần này sẽ được cộng điểm hoặc được xem xét ở mức điểm cao hơn trong rubric.

Nhóm có thể chọn một trong các hướng sau:

- **Quantization:** giảm độ chính xác số học của trọng số hoặc phép tính, ví dụ FP32 sang INT8 hoặc dynamic quantization.
- **Pruning:** loại bỏ một phần trọng số/kênh ít quan trọng để giảm kích thước mô hình.
- **Knowledge Distillation:** dùng mô hình lớn làm teacher và huấn luyện mô hình nhỏ hơn làm student.
- **Lightweight model:** thay mô hình lớn bằng mô hình nhẹ hơn như MobileNetV2, EfficientNet-B0, ShuffleNet hoặc mô hình tự thiết kế nhỏ hơn.

Nếu thực hiện model compression, nhóm cần có bảng so sánh:

Phiên bản	Accuracy/F1	Kích thước	Inference time	Nhận xét
Mô hình gốc				
Mô hình ONNX				
Mô hình nén				

Điểm quan trọng không phải chỉ là làm mô hình nhỏ hơn, mà là sinh viên phải phân tích được sự đánh đổi giữa **độ chính xác, kích thước mô hình, tốc độ suy luận và khả năng triển khai thực tế**.

7 Yêu cầu về báo cáo cuối kỳ

Báo cáo cuối kỳ cần trình bày theo cấu trúc sau:

Báo cáo bài tập lớn CSC4005

1. Giới thiệu bài toán

2. Dataset và tiền xử lý

3. Baseline

4. Mô hình học sâu đề xuất

5. Thiết lập thí nghiệm

6. Kết quả và so sánh

7. Phân tích lỗi

8. ONNX deployment và kiểm thử inference

9. Model compression nếu có

10. Demo hệ thống

11. Hạn chế

12. Kết luận và hướng phát triển

Trong phần kết quả, nhóm cần có tối thiểu:

- Bảng so sánh các runs;
- Link W&B project/report;
- Confusion matrix hoặc biểu đồ metric phù hợp;
- Ảnh/charts minh họa kết quả;
- Phân tích 10 mẫu dự đoán sai;
- Mô tả quy trình export ONNX và kiểm thử `onnxruntime`;
- Bảng so sánh mô hình gốc và mô hình ONNX về kết quả dự đoán, kích thước mô hình và thời gian suy luận.

8 Rubric chấm điểm đề xuất

Thành phần	Điểm	Mô tả
Dataset và tiền xử lý	1.5	Nguồn dữ liệu rõ ràng, tải/đọc dữ liệu đúng, chia train/validation/test hợp lý, tiền xử lý phù hợp với kiểu dữ liệu.
Thiết kế bài toán	1.0	Bài toán rõ ràng, đầu vào/đầu ra/nhãn/metric phù hợp, giải thích được tính ứng dụng của đề tài.
Baseline	1.0	Có baseline chạy được, kết quả được ghi nhận và dùng để so sánh với mô hình học sâu.
Mô hình học sâu	1.5	Có mô hình học sâu phù hợp, huấn luyện đúng quy trình, có giải thích lựa chọn kiến trúc và cải tiến so với baseline.
W&B và thí nghiệm	1.2	Có tối thiểu 5 runs, log đầy đủ hyperparameters/metrics, có so sánh thí nghiệm, có best model hoặc checkpoint.
Đánh giá và phân tích lỗi	1.3	Metric đúng, có confusion matrix hoặc phân tích tương đương, phân tích ít nhất 10 mẫu lỗi, chỉ ra nguyên nhân và hướng cải thiện.
ONNX deployment bắt buộc	1.0	Có file <code>.onnx</code> , có script suy luận bằng <code>onnxruntime</code> , kiểm thử trên mẫu test, so sánh kết quả mô hình gốc và ONNX, đo inference time.
Demo và báo cáo	1.0	Demo chạy được, báo cáo rõ ràng, trình bày tốt, có phần ONNX deployment và hướng phát triển.
Tái lập kết quả	0.5	Có README, requirements, hướng dẫn chạy lại code, cấu trúc thư mục rõ ràng và có hướng dẫn export/chạy ONNX.

Điểm cộng nâng cao: Nhóm thực hiện model compression tốt có thể được cộng tối đa **1.0 điểm**, nhưng tổng điểm cuối cùng không vượt quá 10.0.

Tiêu chí nâng cao	Điểm cộng	Mô tả
Model compression	+0.5	Có áp dụng ít nhất một kỹ thuật như quantization, pruning, knowledge distillation hoặc dùng lightweight model.
So sánh trước–sau compression	+0.5	Có bảng so sánh accuracy/F1, kích thước mô hình, inference time và phân tích trade-off.

9 Lịch trình gợi ý

Phase	Công việc
Phase 1	Chọn đề tài, lập nhóm, tìm dataset, đọc tài liệu liên quan, xác định baseline.
Phase 2	Hoàn thiện pipeline tải dữ liệu, tiền xử lý, tạo train/dev/test split.
Phase 3	Xây baseline và chạy baseline run trên W&B.
Phase 4	Xây mô hình học sâu đầu tiên.
Phase 5	Chạy các thí nghiệm thay đổi hyperparameters, preprocessing hoặc model.
Phase 6	Đánh giá mô hình, phân tích lỗi, hoàn thiện W&B report.
Phase 7	Export mô hình sang ONNX, kiểm thử onnxruntime, đo inference time; nhóm khá/giỏi có thể thực hiện model compression.
Phase 8	Xây demo, viết báo cáo, nộp sản phẩm, trình bày và phản biện.

10 Yêu cầu nộp bài

Mỗi nhóm nộp tối thiểu:

- Link GitHub repository;
- File README.md;
- Source code đầy đủ;
- File requirements.txt;
- File cấu hình thí nghiệm trong thư mục configs/;
- File kết quả metrics_summary.csv hoặc metrics_summary.json;
- File dự đoán test_predictions.csv;
- File reports/experiment_log.md;
- File reports/error_analysis.md;
- File mô hình gốc, ví dụ outputs/models/best_model.pt;
- File mô hình ONNX, ví dụ outputs/models/best_model.onnx;
- Script src/export_onnx.py;
- Script src/infer_onnx.py;
- File reports/onnx_deployment.md;
- File reports/compression_report.md nếu nhóm thực hiện model compression;
- Link W&B project/report;
- Demo app hoặc notebook demo;
- Báo cáo cuối kỳ dạng PDF.

11 Những lỗi rất hay gặp

- Không có dataset rõ ràng hoặc không ghi nguồn dữ liệu;
- Chỉ dùng mô hình có sẵn mà không có baseline và không có thí nghiệm;
- Không chia train/validation/test đúng cách;
- Dùng test set để tuning nhiều lần;

- Có W&B nhưng không log hyperparameters;
- Chỉ báo accuracy, không phân tích macro-F1 hoặc lỗi theo nhãn;
- Không phân tích lỗi thật mà chỉ nhận xét chung chung;
- Demo chạy được nhưng không có đánh giá định lượng;
- Không export được mô hình sang ONNX hoặc không chạy kiểm thử bằng `onnxruntime`;
- Có file ONNX nhưng không so sánh kết quả với mô hình gốc;
- Có model compression nhưng không báo cáo trade-off giữa accuracy, kích thước và tốc độ;
- Báo cáo không chỉ ra hạn chế của dữ liệu và mô hình;
- Không cung cấp hướng dẫn chạy lại code.

12 Câu hỏi thảo luận cuối kỳ

Mỗi nhóm chuẩn bị trả lời các câu hỏi sau khi bảo vệ:

1. Dataset của nhóm có phù hợp với bài toán không? Vì sao?
2. Dataset có hạn chế gì?
3. Baseline của nhóm là gì? Mô hình học sâu có vượt baseline không?
4. Vì sao nhóm chọn kiến trúc mô hình này?
5. Metric chính có phản ánh đúng chất lượng hệ thống không?
6. W&B giúp nhóm rút ra điều gì từ các thí nghiệm?
7. Mô hình sai nhiều nhất ở loại mẫu nào?
8. Nếu có thêm 2 tuần, nhóm sẽ cải thiện dữ liệu, mô hình hay đánh giá trước?
9. Nhóm export ONNX như thế nào? Kết quả PyTorch và ONNX có tương đương không?
10. Inference time của mô hình gốc và mô hình ONNX khác nhau như thế nào?
11. Nếu có thực hiện model compression, nhóm đã đánh đổi gì giữa accuracy, kích thước mô hình và tốc độ suy luận?
12. Demo hiện tại có thể triển khai thực tế chưa? Vì sao?
13. Hạn chế lớn nhất của hệ thống hiện tại là gì?

13 Nguồn dữ liệu và tài liệu tham khảo gợi ý

- W&B Experiment Tracking: <https://docs.wandb.ai/models/track>
- W&B Artifacts: <https://docs.wandb.ai/guides/artifacts/>
- ONNX Documentation: <https://onnx.ai/>
- ONNX Runtime Documentation: <https://onnxruntime.ai/docs/>
- PyTorch ONNX Export: <https://pytorch.org/docs/stable/onnx.html>
- PyTorch Quantization: <https://pytorch.org/docs/stable/quantization.html>
- PlantVillage Dataset: <https://www.kaggle.com/datasets/emmarex/plantdisease>

- UNSW-NB15: <https://research.unsw.edu.au/projects/unsw-nb15-dataset>
- CICIDS2017: <https://www.unb.ca/cic/datasets/ids-2017.html>
- UCI HAR Dataset: <https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>
- EuroSAT: <https://github.com/phelber/eurosat>
- Fakeddit: <https://github.com/entitize/Fakeddit>
- HAM10000: <https://www.kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000>
- MVTec AD: <https://www.mvtec.com/company/research/datasets/mvtec-ad>
- TrashNet: <https://github.com/garythung/trashnet>
- Garbage Classification Dataset: <https://www.kaggle.com/datasets/asdasdasdasdas/garbage-classification>
- MIT-BIH Arrhythmia Database: <https://physionet.org/content/mitdb/1.0.0/>
- EMBER Dataset: <https://github.com/elastic/ember>

14 Kết luận

Bài tập lớn này yêu cầu sinh viên thực hành một quy trình học sâu có thể tái lập:

Dataset → Baseline → Mô hình học sâu → W&B Tracking → Evaluation → Error Analysis → Demo

Một đề tài tốt không chỉ có điểm số cao, mà còn phải có thí nghiệm rõ ràng, so sánh thuyết phục, phân tích lỗi thật và sản phẩm có thể chạy lại. Đây là cơ hội để sinh viên chuyển từ việc học từng mô hình riêng lẻ sang tư duy xây dựng một hệ thống học sâu hoàn chỉnh.